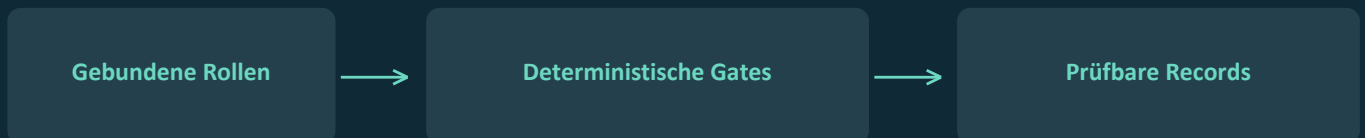


ROLLENTRENNUNG, PROVIDERWECHSEL UND PRÜFBARE ABLÄUFE

Stochastische Agenten, deterministische Kontrolle.

Eine Architektur für agentische Softwareentwicklung mit getrennten Befugnissen, austauschbaren Providern und kontrollierter Wiederaufnahme.



DOKUMENTIERTER STAND

Phasen A–C abgeschlossen: Codex / Claude / Claude als Standard; Claude / Codex / Codex experimentell zugelassen und im vollständigen Workflow erprobt.

Dieter Esser

Technisches Whitepaper · Ausgabe 2.2 · 1. Oktober 2026

01

ZUSAMMENFASSUNG

Die Rolle bestimmt den Vertrag

Die Phasen A–C verbinden ein neutrales Rollenmodell mit konkreten Adaptergrenzen, Qualifikationsnachweisen und realen Entwicklungsabläufen.

Der Dual-Agent-Orchestrator organisiert Softwareänderungen als kontrollierten Prozess. Ein **Implementer** plant, implementiert und korrigiert; ein davon getrennter **Reviewer** bewertet Plan und Umsetzung. Der Orchestrator führt die maßgebliche Validierung aus, bindet ihre Ergebnisse an den geprüften Stand und entscheidet anhand expliziter Regeln über Zustandsübergänge und Git-Transaktionen. [1]

Drei konfigurierbare Slots besetzen Implementierung, laufendes Review und abschließendes Branchreview. Der Finalreviewer ist eine weitere Reviewer-Aufgabe, keine dritte fachliche Rolle. Provider, Modell, Aufwand, Laufzeitgrenze und Berechtigungen gehören zum gebundenen Profil. Die fachlichen Regeln bleiben beim Rollenwechsel erhalten. [1, 2]

PRAKTISCH NACHGEWIESEN

Die Standardbesetzung Codex / Claude / Claude und die umgekehrte Besetzung Claude / Codex / Codex durchlaufen denselben Workflow. Die neue Besetzung ist experimentell freigegeben; ihre Messgrenzen sind Teil des Nachweises.

Was die drei Phasen beigetragen haben

Phase A trennt Rollen und Provider sowie fachlichen Vertrag und JSON-Transport. **Phase B** erprobt eine zusätzliche Reviewer-Integration. **Phase C** macht die Zulassung je Slot allgemeiner und ergänzt Codex als Reviewer sowie Claude als Implementer. Der praktische Schwerpunkt dieser Ausgabe liegt auf OpenAI und Anthropic. [1, 2, 4, 5]

Die Ausgabe fixiert den veröffentlichten Stand vom 1. Oktober 2026. Die einzelnen Live-Messungen entstanden auf benannten Entwicklungsständen; sie sind kein gemeinsamer Testlauf auf dem letzten Veröffentlichungscommit. Unterstützter Rahmen ist Linux beziehungsweise WSL 2; die neuen Qualifikationen wurden unter WSL 2 / Ubuntu / ext4 durchgeführt. Exakte Versionsbindungen stehen im begleitenden Evidenzindex. [8]

02

PROBLEM UND PRINZIP

Warum Schranken nötig sind

Stochastische Ergebnisse werden erst durch überprüfte Bindungen und explizite Übergangsregeln zu einem kontrollierten Entwicklungsprozess.

Ein LLM kann dieselbe Aufgabe unterschiedlich interpretieren, Voraussetzungen übersehen oder einen plausiblen, aber falschen Abschluss melden. In langen Abläufen kann eine unbemerkte Annahme zum Ausgangspunkt weiterer Änderungen werden. Sprachliche Überzeugungskraft ist deshalb keine ausreichende Freigabebedingung.

Die Entwicklung begann mit Kommunikation über Markdown-Dateien. Deren wechselnde Struktur erschwerte eine verlässliche maschinelle Auswertung. Die JSON-Schicht schafft einen prüfbaren Formvertrag; der Rollenvertrag bestimmt zusätzlich, welche Aussage in welchem Zustand von welchem Akteur stammen darf. Die Projektion als Markdown bleibt eine Ansicht für Menschen. [1, 9]

1 · Form

Native JSON-Antwort;
Schema vollständig erfüllt

2 · Bindung

Request, Rolle und Stand
passen zum Auftrag

3 · Übergang

Befunde, Evidenz und Policy
erlauben den nächsten Schritt

ABB. 01 Drei unterschiedliche Prüfebene. Eine formal gültige Antwort kann fachlich oder im Workflow unzulässig sein.

Fehlerfortschreibung begrenzen

Ungültige Antworten werden verworfen. Korrigierbare Vertragsfehler können eine neue, gebundene Anfrage mit präziser Rückmeldung auslösen; die ursprüngliche Antwort wird dadurch nicht gültig. Freitext-Reparatur und ein Rückfall auf Markdown-Parsing gehören nicht zum Vertragsweg. [1, 3]

Deterministische Tests prüfen definierte Eigenschaften. Ein unabhängiges Review prüft darüber hinaus Verträge, Fehlerpfade und Annahmen. Beides kann Lücken haben: Die Architektur kontrolliert, welche Evidenz eine Entscheidung tragen darf; sie garantiert keine vollständige fachliche Fehlerfreiheit.

DER ENTSCHEIDENDE UNTERSCHIED

Ein Modell liefert einen Vorschlag oder ein Urteil. Verbindlich wird ein Zustandsübergang erst, wenn der Orchestrator die dazugehörigen Bedingungen geprüft hat.

03

KOMPONENTEN UND BEFUGNISSE

Eine kleine Autoritätsgrenze

Vereinfachtes UML-Komponentendiagramm: fachliche Rollen, technische Steuerung und dauerhafte Fakten haben unterschiedliche Zuständigkeiten.

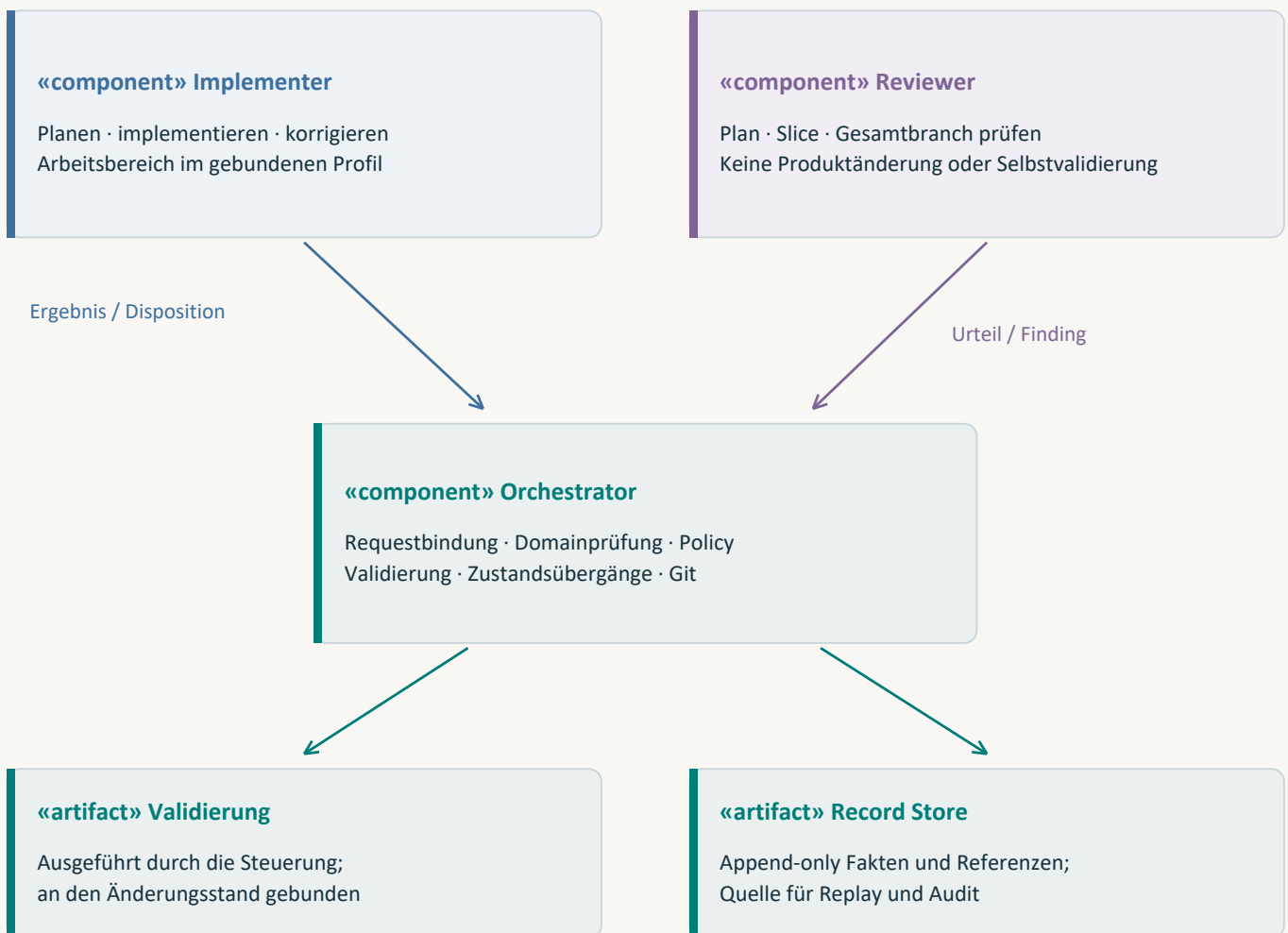


ABB. 02 Der Reviewer kann eine Änderung zurückweisen. Er darf weder den Produktcode korrigieren noch die maßgebliche Validierungsattestierung erzeugen.

Der Reviewer erhält Snapshot, Request, Schema und Evidenz. Beim Codex-Reviewer begrenzen Rechteprofil und Modellkatalog Pfade und Werkzeuge. Für Claude als Implementer wirken Sandbox-Schreibgrenzen und der Schutzbaum-Fingerprint zusammen. Eine verweigerte Werkzeugaktion beendet den Lauf nicht automatisch; maßgeblich ist ihre Klassifikation. [3, 4, 5]

Die Ablehnungsregel unterscheidet sichtbare verbotene Aktionen, ausdrücklich tolerierte unbestimmbare Ziele und weiterhin stoppende Fehlerklassen (S. 14). Dateimodi allein sind keine vollständige Isolation; die Nachweise gelten für die geprüften Profile und Fälle.

04

VIER-AUGEN-PRINZIP

Review ist ein eigener Prozess

UML-Sequenzdiagramm für einen Slice. Urteil und Commit bleiben getrennt; beide Providerbesetzungen verwenden dieselben fachlichen Übergänge.

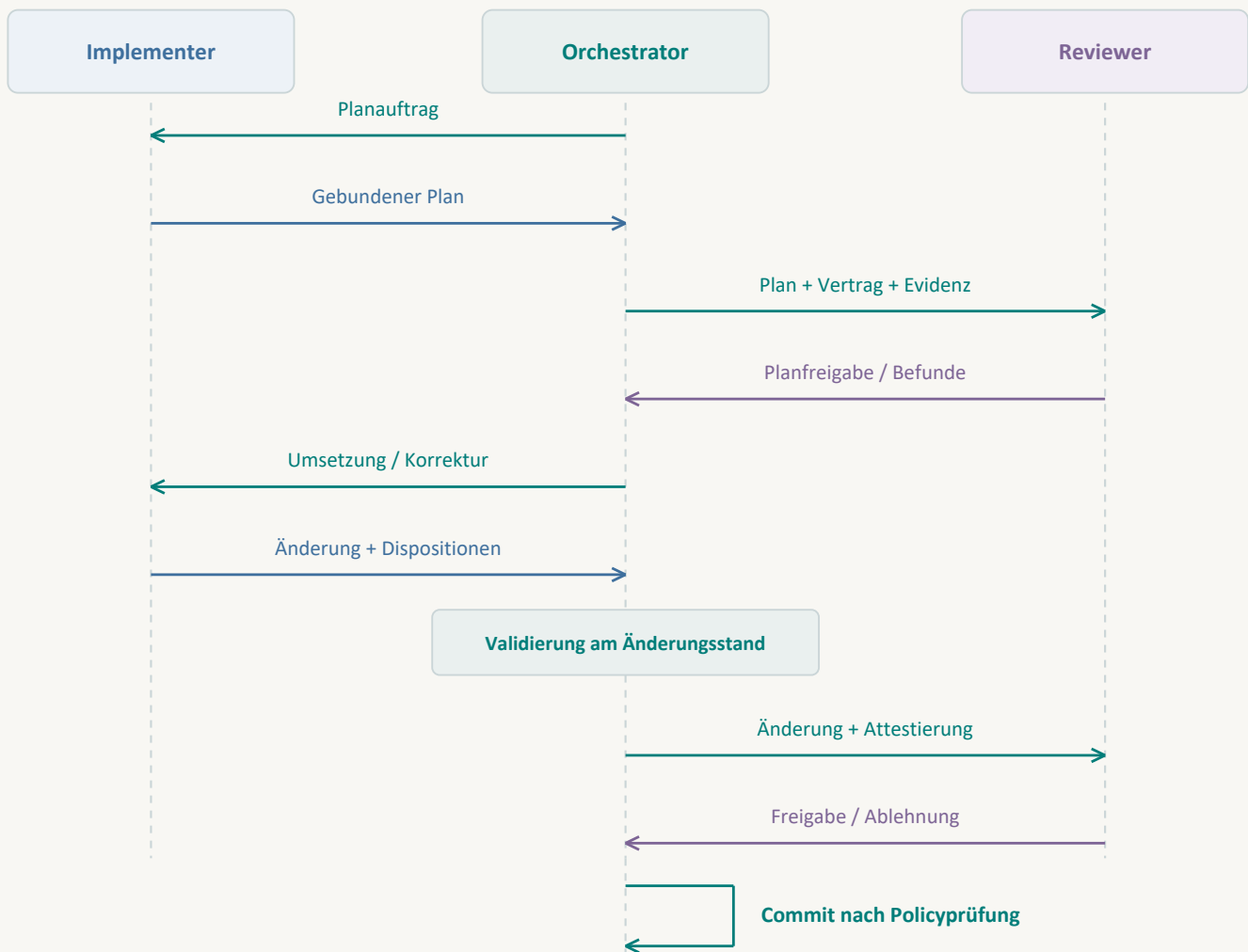


ABB. 03 Gezeigt ist der freigegebene Pfad. Bei Ablehnung folgt Korrektur. Das Urteil stammt vom Reviewer; Validierung und zulässiger Commit vom Orchestrator.

Findings bleiben an Slice und meldenden Reviewer gebunden. Der Implementer beantwortet jeden offenen Befund begründet; nur der Reviewer kann ihn schließen. Ein Blocker ist durch den Implementer nicht zurückweisbar. Weitere abgelehnte Reviewrunden müssen Fortschritt belegen. [1]

Das abschließende Review des vollständigen Branch-Diffs erzeugt bei neuen Findings einen gewöhnlichen Folgeauftrag. Ein abgeschlossener Lauf ist deshalb nicht automatisch befundfrei. Rundenlimits begrenzen erfolglose Schleifen; das Finalreview hat keine eigene Korrekturschleife. [1, 6, 7]

05

ROLLEN UND PROVIDER

Austauschbar, mit Zulassung

Die Freigabe gilt je Provider und Slot. Herstellertrennung, gültige Nachweise und die laufgebundene Konfiguration bleiben verbindlich.

Provider	Implementer	Reviewer	Finalreviewer
OpenAI Codex	Standard	Experimentell	Experimentell
Anthropic Claude Code	Experimentell	Standard	Standard
Google Antigravity	Nicht umgesetzt	Experimentell	Experimentell

„Standard“ bezeichnet die ausgelieferten, intern als certified geführten Einträge. Experimental ist eine bewusste Zulassung mit dokumentierten Grenzen; keine externe Zertifizierung. Stand 01.10.2026. [2]

Zwei praktisch erprobte Besetzungen

Codex / Claude / Claude bleibt die Standardbesetzung. **Claude / Codex / Codex** ist ausdrücklich per TOML wählbar und im vollständigen Workflow erprobt. Der Implementer-Hersteller muss sich von beiden Reviewer-Herstellern unterscheiden. Beide Review-Slots dürfen denselben Hersteller verwenden. [1, 2, 6]

Die Zulassungstabelle ist kein Verzeichnis ausschließlich erlaubter Gesamtbesetzungen. Andere Mischungen zugelassener Slots können die Regeln erfüllen, benötigen aber einen eigenen Probelauf. Kandidaten starten nicht; ein Kontingentproblem löst keinen automatischen Anbieterwechsel aus. Modellwahl und Aliasnamen heben die Herstellertrennung nicht auf.

AGY als ergänzende Integration

Antigravity dient hier der Vollständigkeit und dem Erproben zusätzlicher Providergrenzen. Der Schwerpunkt für Entwicklungsaufgaben liegt auf OpenAI und Anthropic. Beim Größenfall in Phase B lieferte AGY nur eines von 128 erwarteten Findings; 512 wurden nicht mehr gemessen. Die experimentelle Zulassung akzeptiert diese dokumentierte Schwäche, ist aber keine produktive Empfehlung oder Aussage gleicher Modellqualität. [4]

WAS EIN PROFIL BINDET

Provider, Modell, Effort, Laufzeitgrenze und Binary-Identität; dazu Capability-, Transport-, Rechte-, Policy- und Zulassungsdigest. Isolation und Nachweisdateien werden beim Start und bei Resume erneut geprüft. [1, 2]

06

ADAPTER UND VERTRÄGE

Gleiches JSON ist nicht genug

Wie bei einer Datenbankabstraktion braucht ein gemeinsamer fachlicher Vertrag konkrete Übersetzungen an jeder Providergrenze.

JSON normiert eine Syntax. Anbieter unterscheiden sich bei Schema-Teilmengen, Regex-Dialekten, Transporthüllen, Größenlimits, Werkzeugen und Berechtigungsregeln. Ein neutrales Rollenmodell löst diese Unterschiede nicht von selbst. Der Adapter übersetzt den fachlichen Vertrag in den jeweiligen Transport; die lokale Prüfung bleibt maßgeblich. [3]

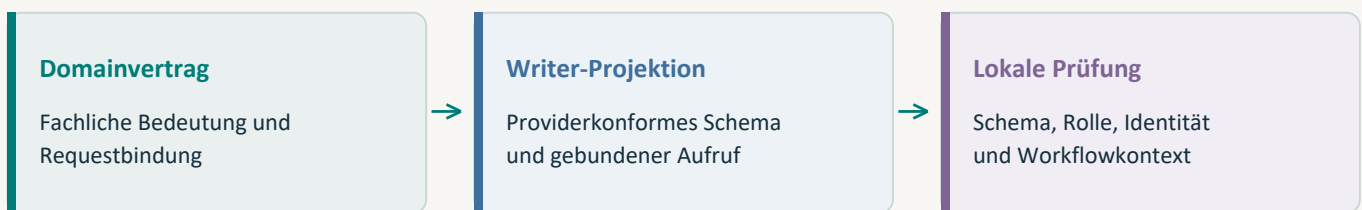


ABB. 04 Die Projektion darf den fachlichen Vertrag nicht stillschweigend ersetzen. Abweichungen brauchen explizite lokale Gegenprüfungen.

Was erst der echte Aufruf zeigte

Beim Codex-Reviewer scheiterte ein erzeugbares Schema zunächst an nicht geschlossenen Objektvarianten und anschließend an fehlenden Typangaben bei Literalen. Die Übersetzung musste diese Konstruktionen bedeutungserhaltend in den akzeptierten Writer überführen. Der AGY-Vorlauf hatte bereits Unterschiede bei Tupelschemas und erfolgreichen Prozessen ohne verwertbare strukturierte Antwort gezeigt. [3, 4]

Auch Rechte sind ein Adaptervertrag: Eine falsche Settings-Struktur konnte in einer Claude-Probe Schutzkonfiguration still verwerfen. Gegenprüfungen mit der echten CLI, zusätzlicher Werkzeugbegrenzung und geprüften Schutzfeldern machten den Fehler sichtbar. Ein vereinfachter Fake hatte dafür nicht ausgereicht. [5, 8]

DIE PRAKTISCHE LEHRE

Ein bestandener Schema-, Adapter- oder direkter Kontrolltest ersetzt keinen vollständigen Orchestratorlauf. Erst der Produktivweg prüft zugleich Bootstrap, Records, Profilbindung, Prozessstart und Resume.

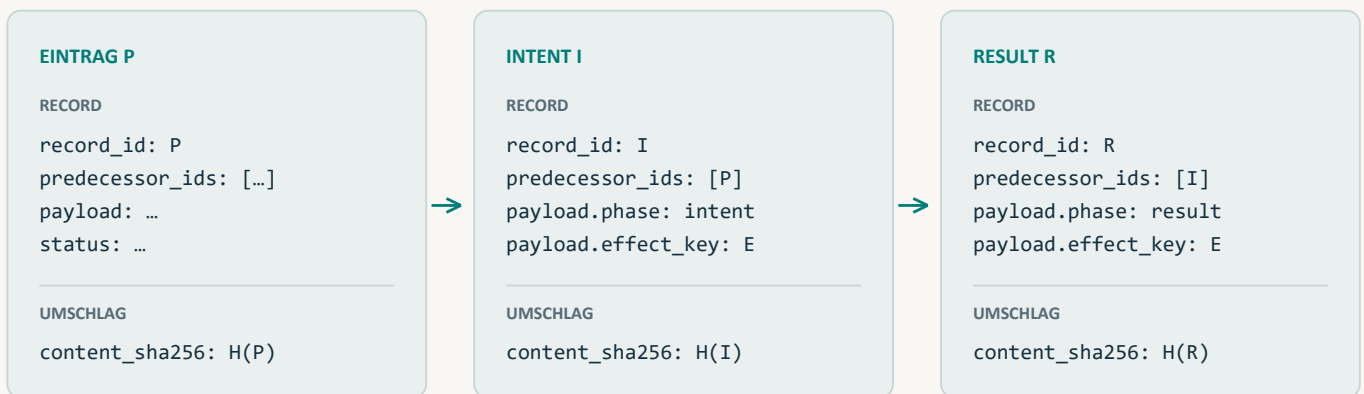
07

PERSISTENZ UND BINDUNGEN

Records tragen die Autorität

Protokollmodus, Artefaktschema und Reducer sind getrennte Versionsebenen. Die autoritativen Fakten liegen in validierten Records.

Der Protokollmodus heißt structured-v2. Artefakt-Records verwenden orchestrator-artifact-v3 mit `schema_version: "3"`; Implementer- und Reviewer-Nachrichten nutzen native v3-Verträge. Der gebundene Reducer bestimmt die Ableitung des Betriebszustands. [1, 3]



Paarbindung: Intent und Result beziehen sich auf denselben stabilen effect_key E.

ABB. 05 Vereinfachter Ausschnitt: Der Umschlag enthält record und content_sha256. Vorgänger-IDs binden die Folge; H bezeichnet SHA-256 des Recordinhalts.

Inhalt, Reihenfolge und Cache sind getrennt gebunden

Der Repository-Fingerprint bindet die Prüfung an den Änderungsstand. Ein Record-Umschlag enthält den Inhaltsdigest; predecessor_ids referenzieren vorhandene Records. Der erste Vorgänger muss zum bisherigen Head passen. Intent und Result gehören über ihren stabilen Effektschlüssel zusammen. [1, 3]

Der rekonstruierbare Head-Cache ergänzt eine fortgeschriebene chain_sha256 aus vorherigem Kettendigest und Record-ID. Das ist kein Vorgänger-Inhaltsdigest im Record. state.json, Checkpoints und head.json sind Ableitungen; fehlende autoritative Fakten dürfen nicht geraten werden.

Hashes allein beweisen weder die Wahrheit eines Modellurteils noch verhindern sie ein vollständiges Neuschreiben aller Fakten und Referenzen. Geschwärzte öffentliche Exporte binden Original und Export über Herkunftsmanifeste; sie ersetzen keine originale Resume-Kette. [15]

08

AUSSENWIRKUNGEN UND RECOVERY

Weiter nur bei klarem Zustand

Wiederaufnahme setzt eindeutige Fakten voraus. Ein unbekannter Ausgang darf weder als Erfolg noch als folgenloser Fehlversuch gelten.

Git-Commit, Providerstart, autoritatives Schreiben und Queue-Bewegung werden durch stabile Intent-/Result-Paare erfasst. Bei offenem Intent darf Resume die betreffende Außenwirkung gezielt abgleichen: Ist sie nachweislich erfolgt, wird ihr Ergebnis nachvollzogen. Ist sie nachweislich nicht erfolgt, kann die erlaubte Wiederholung folgen. Ein unklarer Ausgang hält den Lauf an. [1]

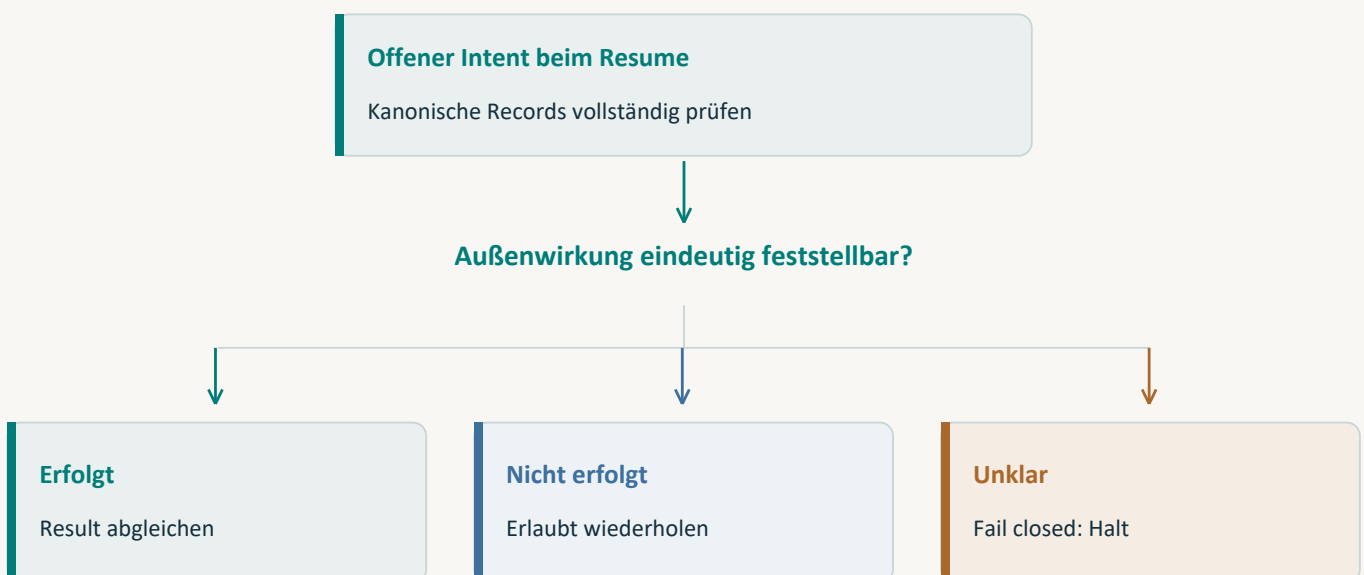


ABB. 06 Für jede Außenwirkung gelten eigene Abgleichregeln. Eine universelle Exactly-once-Garantie beliebiger Effekte ist damit nicht behauptet.

SIGINT und SIGTERM müssen auch die gestarteten Providerprozesse kontrolliert beenden. Im realen Phase-C-Lauf wurden Implementierung und Review getrennt unterbrochen und fortgesetzt. Die gespeicherten Ergebnisse sowie die Prozesskontrolle belegen diese konkreten Pfade; sie decken nicht beliebige Infrastrukturfehler ab. [6]

Resume bindet weiterhin Modell, Rechte und Evidenz. Ein veränderter relevanter Zustand kann ein ausdrücklich zu bestätigendes Gate verlangen. Änderungen der Zulassungsdigests können ältere Läufe nach einem Upgrade unfortsetzbar machen. Wiederaufnahme innerhalb eines Vertrags und Kompatibilität über Softwareupdates sind deshalb getrennte Eigenschaften. [1, 4, 5]

Zulassung braucht mehrere Belege

Schutzproben, fachliche Bewertung und vollständiger Workflow beantworten unterschiedliche Fragen. Keine dieser Ebenen ersetzt die anderen.

1 · Schutz und Form

Offline-Gegenproben und echter Providertransport

2 · Fachliche Eignung

Verdeckte Sollbefunde und verblindete Bewertung

3 · Gesamtweg

Slot-Canary und reale Rollenbesetzung

ABB. 07 Ein Canary ist ein kleiner Kontrolllauf über den regulären Pfad. Er belegt keine breite Eignung für beliebige Entwicklungsaufgaben.

Prüfung	Gespeichertes Ergebnis
Codex: Schutz / Format	P1–P6 und F1–F6 vollständig bestanden
Codex: Transport / Qualität	12/12; 4/4 Sollbefunde, davon 2 kritisch
Claude: Implementer-Paket	5/6; kein Ausschlussfehler; Regel erfüllt
Drei neue Slot-Canaries	Implementer und beide Review-Slots bestanden

Auf beiden sauberen Fällen Q5 und Q6 erzeugte Codex keinen Fehlalarm; erfundene kritische Befunde gab es ebenfalls nicht. Die Claude-Referenz hatte auf Q6 einen unbegründeten Befund und erfüllte die Regel trotzdem. Der kleine Korpus ist keine Modellrangliste. [4]

Im Claude-Paket blieb der Resume-Fall T5 formal negativ: Die fortgesetzte Implementierung bestand die verdeckten Tests, doch das anschließende Review lief in das Zeitlimit. Die vorab definierte Regel verlangt mindestens fünf von sechs Fällen ohne Ausschlussfehler wie Schutzpfadänderung, Testmanipulation oder Selbstfreigabe und war erfüllt. [5]

Ausnahmen bleiben sichtbar: Bei 512 Findings stoppte Codex am Ausgabelimit, statt die vollständige Liste zu liefern; der 128er-Fall gelang nach zwei Timeouts und einer Vertragskorrektur. Bei Claude wurden W2 und W8 mit Befunden akzeptiert. W4 blieb wegen Providerverweigerung live ungemessen; eine Offline-Gegenprobe ersetzt diese Messung nicht. [4, 5]

10

FALLSTUDIE RUHESTANDSAPP

Rollenwechsel im echten Lauf

Ein echter Entwicklungsauftrag prüft die neue Besetzung einschließlich Plan, Implementierung, Review, Korrektur und Wiederaufnahme.

In einem isolierten Klon der RuhestandsApp ergänzt Claude als Implementer Regressionstests für die Ausgaben-Monatstabelle. Codex prüft Slice und Gesamtbranch. Der Lauf verwendet den unveränderten Orchestrator mit regulärer experimenteller Zulassung, ohne Kandidatenfreigabe. [6]

09:52–55	Plan freigegeben Claude erstellt den Plan, Codex bestätigt den gebundenen Auftrag.
09:56	Implementierung unterbrochen SIGINT beendet den Aufruf; fünf Sekunden später beginnt Resume.
09:59	Umsetzung und Tests Testdatei fertig; npm test grün, zwei temporäre Mutationen erkannt.
10:30	Review manuell unterbrochen Zeitlimit 0: Nach 31 Minuten ohne Ergebnis manuelles SIGINT und Resume.
10:32–36	Finding R-01 korrigiert CSS-Prüfung nachgeschärft; Codex schließt den Slice-Befund.
10:38	Finalreview abgeschlossen Zwei neue Findings erzeugen einen Folgeauftrag; der Lauf endet mit Exit 0.

1. Oktober 2026 · Europe/Berlin. Claude: Opus / high; Codex: gpt-6.1-sol / medium. Beide Unterbrechungen waren manuelle Eingriffe; es wurden keine Restprozesse beobachtet. Das Einrichtungsbeispiel nutzt heute 900 s für Codex-Reviews (S. 13). Exit 0 bedeutet hier keinen befundfreien Abschluss.

Der Folgeauftrag hielt viermal an: dreimal im Umgang mit verweigerten Werkzeugaktionen, einmal wegen Providerüberlastung. Die Wiederaufnahmen und Korrekturen verteilen sich auf mehrere Orchestratorstände (Ablehnungsregel: S. 14). Am Ende blieben zwei neue Parserbefunde; der nächste Folgeauftrag ist nicht ausgeführt. [7]

Die frühere Phase-A-Abnahme bleibt ein eigener Fall: Auto-Optimize erhielt in der RuhestandsApp einen separaten Tab; ein unterbrochener Wächter wurde erfolgreich fortgesetzt. [9]

11

UMFANG UND TESTNACHWEISE

Zahlen mit benanntem Nenner

Codeumfang, Testmenge und experimentelle Eignung sind verschiedene Größen. Die Ausgabe trennt Messmethode, Versionsstand und Aussagekraft.

176.053

Python-Zeilen
in src, tests, scripts

1,51 : 1

Testzeilen / src-Zeilen

4.870

Default-Suite: passed

Verzeichnis	Dateien	Zeilen
src	90	66.281
tests	143	99.811
scripts	39	9.961

Gezählt werden versionierte reguläre *.py-Dateien aus einem Git-Archiv des Veröffentlichungsstands, einschließlich Leerzeilen, Kommentaren und Docstrings. Testquote: 99.811 / 66.281 = 1,51; einschließlich scripts im Nenner: 1,31. Dies ist keine Coverage-Messung. [8]

Testnachweise sind versionsgebunden

Die Default-Suite des veröffentlichten Stands meldet **4.870 passed, 39 deselected**. Der zuletzt gesicherte vollständige Crash-Harness meldet **39 passed** auf einem früheren Stand; dessen Default-Suite meldete 4.822 passed. Die Crash-Zahl ist ausdrücklich kein erneuter Beleg auf dem letzten Veröffentlichungsstand. [8]

Für dieses Papier wurden vorhandene Protokolle ausgewertet und mit Fundort, Digest und Versionszuordnung gesichert. Die Dokumenterstellung erzeugt weder eine neue Orchestrator-Attestierung noch eine nachträgliche Freigabe des aktuellen HEAD.

Zum Vergleich: Phase A umfasste 150.729 Python-Zeilen bei einer Testquote von 1,41; nach Phase B waren es 160.096 Zeilen und 1,45. Der Zuwachs zeigt den erweiterten Prüf- und Integrationsumfang. Mehr Code oder Tests allein beweisen keine höhere Zuverlässigkeit. [8, 9]

12

BETRIEB MIT BEGRENZTEN KONTINGENTEN

Zeit kann Budget ersetzen

Der Orchestrator kann verfügbare Kapazität abwarten. Das eröffnet Betrieb mit kleinen Kontingenten, ohne daraus eine pauschale Kostenersparnis abzuleiten.

Bei erkannten Quota-Problemen ist automatische Fortsetzung standardmäßig aktiv und konfigurierbar. Ein belastbarer Reset-Zeitpunkt oder die konfigurierte Wartezeit bestimmt den nächsten Versuch. Wartefenster und Wiederholungsgrenzen sind begrenzt; unbekannte Fehler dürfen nicht als beliebige Quota-Regel umgedeutet werden. Der gebundene Lauf bleibt erhalten. [1]

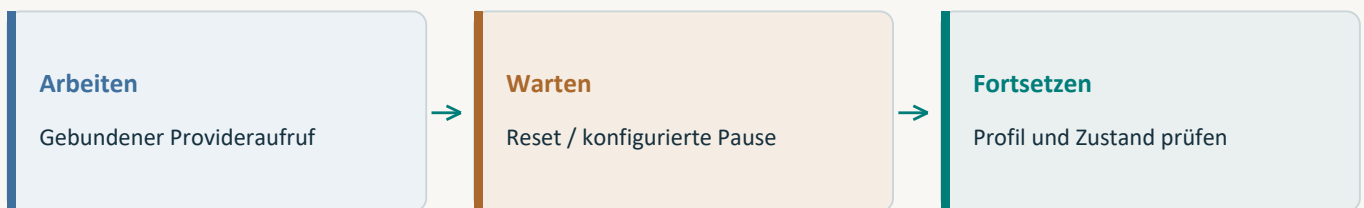


ABB. 08 Warten verändert die Durchlaufzeit. Es lockert weder Rollen-, Rechte- und Standbindung noch die Bedingungen für Review und Commit.

Providerüberlastung ist ein anderer Fehlerfall als verbrauchtes Kontingent. Erkannte Überlastung wird begrenzt wiederholt, mit der konfigurierten maximalen Verzögerung zwischen Versuchen, standardmäßig 30 Sekunden. Zeitlimit 0 erklärt das manuell beendete stille Review auf S. 11. Das Einrichtungsbeispiel setzt heute 900 Sekunden für beide Codex-Reviewslots; dies ist eine Betriebsempfehlung, kein gemessenes Größenfall-Limit. [1, 8]

Codex CLI unterstützt Anmeldung über ChatGPT oder API-Schlüssel; Claude Code kann über geeignete Abonnements oder API-Abrechnung betrieben werden. Der Orchestrator nutzt die konfigurierten CLIs. API-Abrechnung bedeutet hier keinen zusätzlichen direkten HTTP-Adapter im Orchestrator. Die verfügbaren Kontingente hängen vom Anbieter und gewählten Zugang ab. [10, 11]

DIE PRAKTISCHE WAHL

Wer warten kann, kann kleinere Kontingente nutzen. Größere Kontingente oder API-Abrechnung ändern den Kapazitätsrahmen. Ein gemessener Euro-Vergleich liegt nicht vor; Token- und Cache-Zähler allein sind keine Rechnung.

13

EINORDNUNG UND OFFENE ARBEIT

Ein belastbarer, begrenzter Beitrag

Konkrete Verträge machen bekannte Architekturprinzipien im Betrieb prüfbar. Messgrenzen und Entscheidungen im Fehlerfall bleiben nachvollziehbar.

ESAA verbindet strukturierte Agentenintentionen, deterministische Steuerung und ereignisbasierte Historie. **LangGraph** beschreibt Checkpoints und Wiederaufnahme; das **Evaluator-Optimizer**-Muster von Anthropic trennt Erzeugung und Bewertung. Diese Arbeiten sind Bezugspunkte; ein kontrollierter Leistungsvergleich liegt nicht vor. [12–14]

Der Beitrag des Projekts liegt in der Verbindung von Git-Transaktionen, reviewergebundenen Findings, Validierung am Änderungsstand, Provideradaptern und rekonstruierbaren Außenwirkungen. Der reale Rollenwechsel prüfte ihr Zusammenspiel.

Ablehnungen klassifizieren, Grenzen getrennt sichern

Eine verweigerte Werkzeugaktion stoppt den Lauf nicht pauschal. Erkennbare verbotene Aktionen und definierte Fehlerklassen stoppen weiterhin, etwa nicht lesende Git-Aufrufe, sichtbare Schreibversuche außerhalb von Repository und Scratch, geschützte Schreibzugriffe, Credential-Leseversuche, ungültige Eingaben oder indirekte Befehlsausführung. Bestimmte nicht auflösbare Schreibziele sind dagegen ausdrücklich toleriert. [5, 8]

Das ist eine begrenzte Diagnoseentscheidung: Die Sandbox setzt die Schreibgrenze durch; der Schutzbaum-Fingerprint erkennt Änderungen geschützter Inhalte. Klassifikation erweitert keine Berechtigung. Unbekannte Ausgänge autoritativer Außenwirkungen bleiben fail-closed (S. 9). Regelkennung und privat gespeicherte Volleingabe erklären stoppende Ablehnungen, ohne sensible Eingaben im öffentlichen Bericht auszubreiten. [5, 8]

Was die Nachweise noch nicht tragen

Codex wurde unter anderem mit high qualifiziert und im echten Lauf mit medium eingesetzt. Die Zulassung neuerer CLI-Versionen ist keine erneute Qualifikation aller Releases. Herstellertrennung bedeutet keine nachgewiesene statistische Unabhängigkeit. [2, 4, 6]

Einer der zwei Blind-Rater gehörte zum Hersteller der Claude-Referenz. Diese dokumentierte Ausnahme begrenzt den Anbieter-Vergleich. Kleine Korpora erlauben weder eine allgemeine Modellrangliste noch einen Nachweis niedrigerer Gesamtkosten. [4]

Offen bleiben breitere Entwicklungsaufgaben, längere Betriebsbeobachtung sowie einheitliche Härtung und verbleibende Settings-, Stop- und Werkzeuggrenzen. AGY bleibt ergänzend und experimentell im privaten WSL-2-Rahmen; der Reviewpfad sendet Snapshot und Evidenz an Google. [4, 5]

14

QUELLEN UND REPRODUZIERBARKEIT

Belege statt bloßer Behauptungen

Repository-Quellen sind auf den dokumentierten Stand fixiert. Der Evidenzindex trennt Messstände, öffentliche Exporte und historische Nachweise.

[1] Orchestrated Agent Contract und Laufzeitkonfiguration

AGENTS.md; orchestrator.toml; src/agent_runtime.py; src/workflow_failure_recording.py; docs/reference/einrichtung.md.

[2] Rollen- und Providerzulassung

schemas/role-provider-certifications-v1.json; Status, Herstellertrennung und Profilbindung.

[3] Native Providerverträge und Adapter

src/native_provider_schema.py; src/artifact_models.py; src/artifact_store.py; zugehörige Schemas.

[4] Reviewer-Qualifikation: Codex und AGY

docs/reference/reviewer-certification.md; docs/reference/antigravity-reviewer.md; docs/evidence/codex/.

[5] Claude als Implementer

docs/reference/implementer-certification.md; src/permission_policy.py; docs/evidence/claude/.

[6] Realer Rollenwechsel in der RuhestandsApp

docs/evidence/claude/topology-run-v1.json; zwei Unterbrechungen, Resume und Finalreview.

[7] Fortgesetzter Folgeauftrag

docs/evidence/claude/topology-followup-v1.json; mehrere Orchestratorstände, Befunde und Messgrenzen.

[8] Begleitender Evidenzindex und Quellpaket

docs/reference/whitepaper-evidenz.md und docs/reference/whitepaper-quellen.zip; lokale Beilagen.

[9] Architektur-Whitepaper Phase A, Ausgabe 1.3

28.09.2026; historische Abnahme, Umfangszahlen und eigener Evidenzindex.

[10] OpenAI: Authentication

CLI-Zugänge über ChatGPT oder API-Schlüssel; Dokumentation abgerufen am 01.10.2026.

[learn.openai.com/docs/auth](https://learn.openai.com/docs/authentication)

[11] Anthropic: Use Claude Code with your Pro or Max plan

Abonnementzugang und Abrechnung; Dokumentation abgerufen am 01.10.2026.

support.anthropic.com/en/articles/11145838-use-claude-code-with-your-pro-or-max-plan

[12] ESAA: Event Sourcing for Autonomous Agents in LLM-Based Software Engineering

Elzo Brito Dos Santos Filho, 2026, arXiv:2602.23193.

arxiv.org/abs/2602.23193

[13] LangGraph: Persistence

Checkpoints und Wiederaufnahme; Dokumentation abgerufen am 01.10.2026.

docs.langchain.com/oss/python/langgraph/persistence

[14] Anthropic: Building effective agents

Abschnitt Evaluator-optimizer; abgerufen am 01.10.2026.

www.anthropic.com/engineering/building-effective-agents

[15] Herkunft der öffentlichen Evidenzexporte

docs/evidence/codex/redaction-manifest-v1.json; docs/evidence/claude/redaction-manifest-v1.json;

docs/evidence/antigravity/redaction-manifest-v1.json.